

Continue developing my existing **NearServe** project.

The project is a **Local Service Booking Mobile Application**.

IMPORTANT:

- Phase 1 has already been completed.
- Do NOT recreate the project from scratch.
- Do NOT delete or replace the existing Splash, Login, or Signup screens.
- Preserve all existing working code.
- Continue from the current project state.
- If a feature already exists, improve/integrate it instead of creating a duplicate.
- Build the remaining application systematically.
- Keep the application runnable after every major change.
- Fix errors before moving to the next feature.

PROJECT NAME

NearServe

Tagline:

"Your Local Services, Just a Tap Away."

PURPOSE

NearServe helps users discover local service providers, filter them based on their requirements, view service details, book services, and track their bookings.

The main user flow is:

Splash → Login/Signup → Home → Discover → Filter → Service Details → Booking → Confirmation → My Bookings

TECHNOLOGY STACK

Frontend:

- React Native
- Expo
- TypeScript

Navigation:

- React Navigation

Backend:

- Node.js
- Express.js

Database:

- MySQL

API:

- REST API

Authentication:

- JWT
- bcrypt password hashing

Development:

- Antigravity
- Git/GitHub

For the current frontend development, use mock data where backend data is not yet available.

Do NOT use Firebase as the primary database.

EXISTING PHASE 1

The project already contains:

1. Splash Screen
2. Login Screen
3. Signup Screen

Preserve these screens and their existing functionality.

The existing navigation should remain functional.

PHASE 2 — HOME SCREEN

Create a professional NearServe Home screen.

Header

Display:

Hello, User! 🙌

Show:

- Current/selectable location
- Notification icon
- Profile icon

Example:

📍 **Coimbatore**

Search

Create a large search bar:

"What service do you need?"

The search should allow users to search for services.

Service Categories

Create a horizontally scrollable category section.

Categories:

- Plumbing
- Electrical
- Cleaning
- AC Repair
- Appliance Repair
- Beauty & Salon
- Painting
- Vehicle Service
- Gardening
- Computer Repair

Each category should have:

- Icon
- Category name

Add:

See All

Nearby Services

Display service provider cards.

Each card should contain:

- Service image/icon
- Service name
- Provider name
- Rating
- Number of reviews
- Distance
- Starting price
- Availability
- View Details button

Example mock data:

Home Plumbing

Provider:

Kumar Plumbing Services

Rating:

4.8 ★

Distance:

1.2 km

Starting price:

₹299

Availability:

Available Today

AC Repair

Provider:

CoolCare Services

Rating:

4.6 ★

Distance:

2.1 km

Starting price:

₹499

Availability:

Available Now

Home Cleaning

Provider:

FreshHome Cleaning

Rating:

4.7 ★

Distance:

1.8 km

Starting price:

₹399

Availability:

Available Today

Electrical Repair

Provider:

PowerFix Electricals

Rating:

4.5 ★

Distance:

2.5 km

Starting price:

₹249

Availability:

Available Today

BOTTOM NAVIGATION

Create a bottom navigation bar:

Home | Discover | Bookings | Profile

Home should be active.

Discover should open Discover.

Bookings should open My Bookings.

Profile should open Profile.

Use suitable icons.

PHASE 3 — DISCOVER SCREEN

Create a dedicated service discovery screen.

Header:

Discover Local Services

Search bar:

"Search for services..."

Add:

Filter

and

Sort

buttons.

Display service providers in a scrollable list.

Each service card should contain:

- Service image
- Service name
- Provider name
- Rating
- Reviews
- Distance
- Starting price

- Availability
- View Details

SEARCH FUNCTIONALITY

Search should work with:

- Service name
- Provider name
- Category

Example:

If user searches:

Plumbing

show plumbing services.

If user searches:

AC

show AC services.

If no result is found, display:

"No services found."

FILTER FUNCTIONALITY

When the user taps Filter, open a professional bottom sheet/modal.

Category

Options:

- All
- Plumbing
- Electrical
- Cleaning
- AC Repair
- Appliance Repair

- Beauty & Salon
- Painting
- Vehicle Service
- Gardening
- Computer Repair

Distance

Options:

- Within 1 km
- Within 3 km
- Within 5 km
- Within 10 km

Price

Allow the user to select:

Minimum Price

Maximum Price

Example:

₹100 – ₹2000

Rating

Options:

- 4.5★ & above
- 4★ & above
- 3★ & above
- Any rating

Availability

Options:

- Available Now
- Available Today
- Available Tomorrow

Buttons:

Clear All

Apply Filters

Filters must actually update the displayed mock service list.

SORT FUNCTIONALITY

When Sort is selected, display:

- Relevance
- Distance
- Rating
- Price: Low to High
- Price: High to Low

The service list should update according to the selected sorting option.

PHASE 4 — SERVICE DETAILS

Create a complete Service Details screen.

When the user selects a service, display:

- Large service image
- Service name
- Provider name
- Rating
- Number of reviews
- Distance
- Starting price
- Description
- Services offered
- Estimated service duration
- Availability

- Service area
- Provider contact information

Add:

Book Now

button.

Add a Reviews section.

Example:

4.8 ★

125 Reviews

Show a few sample reviews.

Do not use real people's personal information.

PHASE 5 — BOOKING SCREEN

When the user clicks:

Book Now

open the Booking screen.

Title:

Book Service

Display selected:

- Service
- Provider
- Price

Allow user to select:

Date

Use a date selector.

Time

Display available time slots.

Example:

- 9:00 AM
- 10:00 AM
- 11:00 AM
- 2:00 PM
- 4:00 PM
- 6:00 PM

Service Location

Allow the user to enter/select:

- Address
- Area
- City
- Pincode

Additional Instructions

Text box:

"Add instructions for the service provider..."

Price Summary

Show:

Service Charge

₹499

Additional Charges

₹0

Total

₹499

Button:

Confirm Booking

Validate required fields before confirmation.

PHASE 6 — BOOKING CONFIRMATION

After successful booking:

Display:

Booking Confirmed! ✓

Show:

- Booking ID
- Service
- Provider
- Date
- Time
- Location
- Total Price
- Booking Status

Initial status:

Pending

Buttons:

View Booking

Back to Home

Generate a unique booking ID.

Example:

NS20260922001

PHASE 7 — MY BOOKINGS

Create:

My Bookings

Use tabs:

Upcoming

Show:

- Pending

- Confirmed
- In Progress

Completed

Show completed services.

Cancelled

Show cancelled bookings.

Each booking card should contain:

- Service
- Provider
- Date
- Time
- Price
- Status
- View Details

PHASE 8 — BOOKING DETAILS

Create a detailed booking screen.

Display:

- Booking ID
- Service
- Provider
- Date
- Time
- Location
- Price
- Additional instructions
- Booking status

Create a visual status timeline:

Booking Placed

↓

Confirmed

↓

Provider Assigned

↓

Service Started

↓

Completed

Only show the relevant stages based on the current status.

For upcoming bookings:

Cancel Booking

button.

For completed bookings:

Rate Service

button.

PHASE 9 — CANCEL BOOKING

When the user chooses Cancel Booking:

Show confirmation:

"Are you sure you want to cancel this booking?"

Buttons:

Keep Booking

Cancel Booking

After cancellation:

Status becomes:

Cancelled

The booking should move to the Cancelled section.

PHASE 10 — PROFILE

Create a professional Profile screen.

Display:

- Profile image/avatar
- User name
- Email
- Phone
- Location

Options:

Edit Profile

Allow editing:

- Name
- Email
- Phone
- Location

My Bookings

Navigate to My Bookings.

Saved Services

Create a saved/favorite services section.

Notifications

Navigate to Notifications.

Help & Support

Display support information.

Terms & Conditions

Display placeholder terms page.

Privacy Policy

Display placeholder privacy page.

Logout

Logout the user and navigate to Login.

PHASE 11 — NOTIFICATIONS

Create a Notifications screen.

Display notifications such as:

- Booking confirmed
- Booking request received
- Provider assigned
- Service reminder
- Booking completed
- Booking cancelled

Example:

Your AC Repair booking has been confirmed.

Show notification time.

Unread notifications should appear visually different.

PHASE 12 — SAVED SERVICES

Allow users to save/favorite services.

Add a heart icon to service cards.

When selected:

♡ → ♥

Saved services should appear under:

Profile → Saved Services

PHASE 13 — BACKEND

After the frontend screens and navigation are working, create a separate backend.

Structure:

nearserve/

frontend/

backend/

The backend should use:

Node.js + Express.js

Create:

server.js

and organized folders:

backend/

├— controllers/

├— routes/

├— models/

├— middleware/

├— config/

└— database/

PHASE 14 — MYSQL DATABASE

Create MySQL database:

nearserve

Create these tables.

users

Fields:

- id
- name
- email
- phone
- password
- location
- created_at

service_categories

Fields:

- id

- name
- description

service_providers

Fields:

- id
- name
- phone
- email
- category_id
- location
- rating
- total_reviews
- starting_price
- description
- availability

services

Fields:

- id
- provider_id
- category_id
- service_name
- description
- price
- duration
- location
- rating

bookings

Fields:

- id
- booking_id
- user_id
- service_id
- provider_id
- booking_date
- booking_time
- service_location
- notes
- price
- status
- created_at

reviews

Fields:

- id
- user_id
- service_id
- rating
- comment
- created_at

notifications

Fields:

- id
- user_id
- title
- message
- is_read
- created_at

saved_services

Fields:

- id
- user_id
- service_id
- created_at

Use suitable primary keys and relationships.

PHASE 15 — AUTHENTICATION API

Create:

POST /api/auth/signup

POST /api/auth/login

GET /api/users/profile

PUT /api/users/profile

Use:

- bcrypt for password hashing
- JWT for authentication
- Authentication middleware

Never store plain-text passwords.

Never put database passwords inside frontend code.

Use .env for sensitive configuration.

PHASE 16 — SERVICE APIs

Create:

GET /api/services

GET /api/services/:id

GET /api/services/search

GET /api/services/filter

GET /api/categories

GET /api/providers

GET /api/providers/:id

The filter API should support:

- Category
- Distance
- Price
- Rating
- Availability

PHASE 17 — BOOKING APIs

Create:

POST /api/bookings

GET /api/bookings

GET /api/bookings/:id

PUT /api/bookings/:id

DELETE /api/bookings/:id

Allow authenticated users to:

- Create booking
- View bookings
- View booking details
- Cancel booking

PHASE 18 — REVIEW APIs

Create:

POST /api/reviews

GET /api/services/:id/reviews

Allow users to review completed services.

Rating should be between:

1 and 5

PHASE 19 — NOTIFICATION APIs

Create:

GET /api/notifications

PUT /api/notifications/:id/read

Return notifications belonging to the logged-in user.

PHASE 20 — FRONTEND API INTEGRATION

Connect the React Native frontend to the Express backend.

Create an API service layer.

For example:

frontend/src/services/api.ts

Do not put API calls directly inside every UI component.

Use reusable functions for:

- Login
- Signup
- Fetch services
- Search services
- Filter services
- Fetch service details
- Create booking
- Fetch bookings
- Cancel booking
- Update profile
- Fetch notifications
- Save service

Initially, if the backend is unavailable, allow the frontend to use mock data without crashing.

PHASE 21 — ERROR HANDLING

Implement proper error handling.

Display friendly messages for:

- Invalid login
- Signup failure
- Network error
- No services found
- Booking failure
- Invalid date/time
- Missing address
- Server unavailable

Do not expose technical errors directly to users.

PHASE 22 — LOADING STATES

Add loading indicators for:

- Login
- Signup
- Service loading
- Search
- Filtering
- Booking
- Profile updates

Use appropriate empty states.

Example:

No services found

Try changing your filters or search terms.

PHASE 23 — UI DESIGN

The entire application should have a consistent NearServe design.

Use:

Primary:

#5B4BDB

Secondary:

#7C6CE8

Background:

#F8F9FC

Text:

#1F2937

Secondary Text:

#6B7280

White:

#FFFFFF

Success:

#22C55E

Warning:

#F59E0B

Error:

#EF4444

Use:

- Rounded cards
- Modern buttons
- Clean typography
- Consistent spacing

- Subtle shadows
- Professional icons
- Smooth scrolling
- Proper keyboard handling
- Responsive mobile layout

Do not make the interface look like a generic website.

It should look like a real mobile service-booking application.

PHASE 24 — REUSABLE COMPONENTS

Create reusable components where appropriate.

Examples:

- CustomButton
- InputField
- SearchBar
- CategoryCard
- ServiceCard
- ProviderCard
- FilterModal
- SortModal
- BookingCard
- RatingStars
- Header
- BottomNavigation
- LoadingIndicator
- EmptyState
- NotificationCard

Avoid unnecessary duplication.

PHASE 25 — PROJECT STRUCTURE

Use a clean structure similar to:

nearserve/

frontend/

- ├─ src/
 - ├─ screens/
 - ├─ components/
 - ├─ navigation/
 - ├─ services/
 - ├─ hooks/
 - ├─ types/
 - ├─ constants/
 - ├─ utils/
 - └─ assets/
- ├─ App.tsx
- └─ package.json

backend/

- ├─ controllers/
- ├─ routes/
- ├─ models/
- ├─ middleware/
- ├─ config/
- ├─ database/
- ├─ server.js
- ├─ .env
- └─ package.json

README.md

PHASE 26 — NAVIGATION FLOW

The complete navigation should be:

Splash

↓

Login

↓

Home

OR

Splash

↓

Signup

↓

Login

↓

Home

Home:

Home

|— Search

|— Categories

|— Nearby Services

|— Discover

|— My Bookings

|— Notifications

|— Profile

Discover:

Discover

↓

Search / Filter / Sort

↓

Service Details

↓

Book Now

↓

Booking

↓

Booking Confirmation

↓

Booking Details

Profile:

Profile

|— Edit Profile

|— My Bookings

|— Saved Services

|— Notifications

PHASE 27 — IMPORTANT FUNCTIONAL FLOW

The following complete flow must work:

User Registration

Signup

↓

Enter details

↓

Validate

↓

Create account

↓

Login

User Login

Enter email/phone

↓

Enter password

↓

Authenticate

↓

Home

Service Discovery

Home

↓

Search

↓

Discover

↓

Apply filters

↓

View matching services

Service Booking

Service Details

↓

Book Now

↓

Select date

↓

Select time

↓

Enter location

↓

Confirm Booking

↓

Booking Confirmation

Booking Tracking

My Bookings

↓

Select booking

↓

View Booking Details

↓

Track Status

PHASE 28 — TESTING

Before considering the project complete, test:

Authentication

- Signup
- Login
- Invalid login
- Logout

Navigation

- Splash → Login
- Login → Home
- Signup → Login
- Home → Discover
- Discover → Service Details

- Service Details → Booking
- Booking → Confirmation
- Home → My Bookings
- Home → Profile

Search

- Search by service
- Search by provider
- Empty search result

Filters

- Category
- Distance
- Price
- Rating
- Availability
- Clear filters
- Apply filters

Sorting

- Relevance
- Distance
- Rating
- Low-to-high price
- High-to-low price

Booking

- Select date
- Select time
- Enter location
- Confirm booking
- View booking

- Cancel booking

Profile

- Edit profile
- Saved services
- Notifications
- Logout

PHASE 29 — ERROR CHECKING

Before finishing:

1. Check all TypeScript errors.
2. Check all import errors.
3. Check navigation errors.
4. Check API errors.
5. Check backend startup.
6. Check database connection.
7. Check frontend startup.
8. Check that existing Phase 1 screens still work.
9. Remove unused imports.
10. Remove unnecessary packages.
11. Make sure no sensitive credentials are hardcoded.
12. Make sure the application does not crash when the backend is unavailable.

If an error occurs, fix it before continuing.

PHASE 30 — FINAL DOCUMENTATION

Create a README.md explaining:

1. Project overview
2. Features
3. Technology stack

4. Frontend setup
5. Backend setup
6. MySQL setup
7. Environment variables
8. API endpoints
9. How to run the application
10. How to test the application

Also explain the final project architecture in simple terms.

MOST IMPORTANT INSTRUCTION

Do not try to blindly perform everything in one operation.

Work through the project in logical stages while preserving the existing work.

Recommended order:

Stage 1

Home + navigation

Stage 2

Discover + search

Stage 3

Filter + sort

Stage 4

Service Details

Stage 5

Booking

Stage 6

Booking Confirmation + My Bookings

Stage 7

Profile + Notifications + Saved Services

Stage 8

Express.js backend

Stage 9

MySQL database

Stage 10

Authentication APIs

Stage 11

Service APIs

Stage 12

Booking APIs

Stage 13

Frontend/backend integration

Stage 14

Testing + error fixing

Do not delete working code from previous stages.

The final NearServe application should provide a complete working demonstration of:

User Registration → Login → Discover Local Services → Search → Filter → Sort → View Service → Book Service → Booking Confirmation → Track Booking → Manage Profile